in_class_lab7b

A new database, Chapter 5 table and data maintenance, then GitHub from VS Code

CIS 4720 Database Management and SQL

Points	10 points
Time	About 80 minutes
Software	VS Code with SQLTools, the MySQL on your lab PC, the OneDrive app, and a web browser for GitHub
Starts from	in_class_lab6. You do not need the database you built then, because this lab rebuilds it
You submit	One thing only: the web address of your GitHub repository
Due	The due date is listed on Canvas LMS

Read this first. Why any of this matters.

Everything you create on a lab PC is deleted the same night. The machine is wiped and handed to the next class. The database you built in lab6 is already gone, and so is anything you saved to the desktop or to Documents.

That is not a problem you can work around by being careful. It is why this lab has you save your work in two places that are not the lab PC:

- **OneDrive**, which copies your folder to the cloud as you work, so it is there when you sit at a different machine or open your laptop at home.
- **GitHub**, which keeps your file with a web address you can hand to anyone, which is how you get credit for this lab.

Neither one happens on its own. You have to sign in to OneDrive on the PC before you start, or your folder is just a folder on a machine that gets erased tonight.

> **Sign in to the OneDrive app, not the website**
>
> You can reach OneDrive two ways, through the app installed on the PC or through a browser at office.com. For this lab, use the **app**.
>
> Here is why. When you create a file or a folder in the browser, it exists in the cloud first, and the app on the PC has to notice and pull it down. That can take a while, and sometimes it does not happen until you force it. So you make a folder in the browser, go looking for it in File Explorer to open it in VS Code, and it is not there. Students lose twenty minutes to this every time.
>
> Going the other way round is reliable. Create the folder in File Explorer under your OneDrive, and the app uploads it for you within seconds. You can watch it happen in the system tray.
>
> So the rule for this lab is simple: **make folders and files on the PC, and let OneDrive push them up.** Never make them in the browser and hope they come down.

Part 1. Save your work somewhere that survives tonight

Step 1. Sign in to the OneDrive app

Look in the system tray at the bottom right of the screen for the OneDrive cloud icon. If it is grey or has a line through it, you are not signed in.

- Click the cloud icon. If it asks you to sign in, use your campus email address and password.
- If there is no cloud icon at all, press the Windows key, type **OneDrive**, and open it from the list.
- Wait until the icon turns into a plain cloud or shows a tick. That means it is connected and syncing.

Do not skip this and come back to it later. If OneDrive is not signed in when you create the folder in step 2, nothing in it gets uploaded.

Step 2. Make the folder, in File Explorer

Open File Explorer. In the left sidebar you will see your OneDrive, which on a campus machine is usually named OneDrive followed by the university name. Click it, then make a new folder called:

```
CIS4720_Lab7b
```

It has to be inside OneDrive. A folder on the desktop or in Documents is not in OneDrive and will not be uploaded.

Step 3. Download the SQL file into that folder

Download **students_db_backup.sql** from Canvas and save it straight into **CIS4720_Lab7b**. If your browser dropped it in Downloads instead, move it across.

Hover over the file in File Explorer and look at the status column. A green tick means OneDrive has it. A pair of circling arrows means it is still uploading, which is fine, just give it a moment.

Part 2. Create the database, open your file, learn to comment

Step 4. Create the database at the command prompt

The PC wiped your database overnight, so there is nothing on this machine for VS Code to connect to. This is the step that caught the class out last time, so do it before you open VS Code at all.

VS Code asks you which database to connect to, and it will not make one for you. Point it at a database that is not there and you get this, with no way forward:

```
ERROR 1049 (42000): Unknown database 'students_db'
```

So make an empty one first, from the command prompt, the same way you worked in lab6. Press the Windows key, type **cmd**, and open Command Prompt. Then type:

```
mysql -u root -p
```

Press Enter and it asks for the password. Type it and press Enter again.

> **Nothing appears while you type the password**
>
> The screen shows nothing at all as you type, no dots and no stars. That is normal and it is deliberate. Type the password straight through and press Enter.
>
> If you see **'mysql' is not recognized** instead of a password prompt, you are not in the right folder. Change into the MySQL program folder first, the same as lab6, then try again:

```
cd "C:\Program Files\MySQL\MySQL Server 8.0\bin"
mysql -u root -p
```

You are in when the prompt changes to **mysql>**. Now create the database and check that it is there:

```
CREATE DATABASE students_db;

SHOW DATABASES;

EXIT;
```

students_db should appear in the list. That is all you need here. Do not create the table or add any rows, because the file you downloaded in step 3 does all of that for you in step 8.

EXIT ends the command prompt session. Leaving the window open is useful, because it is handy for checking your work against later.

Step 5. Open the folder in VS Code and connect

Open VS Code, then **File**, then **Open Folder**, and choose **CIS4720_Lab7b**. Click **students_db_backup.sql** in the Explorer panel on the left to open it.

Then connect to MySQL. Click the database icon in the left bar, then your saved connection. On a wiped PC that connection is gone too, so add it again with server **localhost**, port **3306**, database **students_db**, user **root**.

This is the database you made in step 4, which is the whole reason step 4 comes first. If you get **unknown database** here, step 4 did not happen, so go back and do it.

Step 6. Commenting out one line

A comment is text the database ignores. You use it to explain what a statement does, and to switch a statement off without deleting it.

For a single line, put two dashes and a space at the front:

```
-- This line is a comment. MySQL ignores it completely.
-- SELECT * FROM STUDENT;    <-- this statement is now switched off

SELECT * FROM STUDENT;       -- a comment can also sit after a statement
```

The fast way, and the one to actually use

You do not have to type the dashes. Select as many lines as you like, then press **Ctrl and the forward slash key**. VS Code puts **--** at the front of every line you selected. Press it again with the same lines selected and it takes them off.

This is the one to use all through this lab. It works on one line or forty, and it never goes wrong.

Step 7. Commenting out a block, and the trap in it

There is a second way, which wraps a whole region between two markers:

```
/* Everything between these two markers is ignored,
   however many lines it runs across.
   SELECT * FROM STUDENT;
*/
```

> **Block comments do not nest, and this will catch you**
>
> You cannot put one block comment inside another. SQL does not look for matching pairs. It starts ignoring at the first **/*** and starts paying attention again at the very next ***/** it finds, whatever was in between.
>
> So if you wrap a region that already contains a block comment, the inner ***/** ends your outer comment early. Everything after it runs when you thought you had switched it off, and the leftover ***/** gives you **error 1064, syntax error near '*/'**.
>
> This is why step 10 has you use **Ctrl and forward slash** rather than block markers. Line comments cannot nest wrongly because every line stands on its own.

Part 3. Build the students_db database

Step 8. Run the file, the first time

Open **students_db_backup.sql** and read the comments at the top. Then run the whole file. In SQLTools, right click in the file and choose **Run Query**, or use the Run link above the statements.

When it finishes you should see four results come back, ending with:

```
students  average_gpa
       5         3.43

Tables_in_students_db  Table_type
STUDENT                BASE TABLE
STUDENT_GPA            VIEW
```

That is the whole starter database built: the schema, the table, five students and the view. Everyone in the room is now in the same place.

Step 9. Run it again on purpose, and read the errors

Run the same file a second time without changing anything. You will get seven errors. Do not fix them yet. Read them.

```
ERROR 1050 (42S01): Table 'STUDENT' already exists
ERROR 1062 (23000): Duplicate entry '1' for key 'PRIMARY'
ERROR 1062 (23000): Duplicate entry '2' for key 'PRIMARY'
ERROR 1062 (23000): Duplicate entry '3' for key 'PRIMARY'
ERROR 1062 (23000): Duplicate entry '4' for key 'PRIMARY'
ERROR 1062 (23000): Duplicate entry '5' for key 'PRIMARY'
ERROR 1050 (42S01): Table 'STUDENT_GPA' already exists
```

Nothing is broken. Every one of those errors is the database protecting your data:

- **1050** says the table is already there. CREATE TABLE will not quietly replace a table that has your data in it, which is exactly what you would want.
- **1062** says the primary key already has a row with that id. The four INSERT statements are trying to add rows that exist, and the primary key refuses, four times.
- **1050** again for the view, for the same reason as the table.

Step 10. Switch off the statements that only needed to run once

This is the point of steps 8 and 9. CREATE and INSERT build things. Once built, they are built, and running them again can only fail. From here on you want to run this file freely without wading through seven errors every time, because by the end of the lab you will be adding to it and running it over and over.

So switch those statements off, without deleting them:

1. Click on the first line of **SECTION 2**, the CREATE TABLE part.
2. Hold **Shift** and click on the last line of **SECTION 4**, the line that reads **FROM STUDENT;** under the CREATE VIEW. That selects sections 2, 3 and 4 together.
3. Press **Ctrl and the forward slash key**. Every selected line gets **--** in front of it and turns grey.
4. Run the file again. Zero errors, and the four read only checks in SECTION 5 still return their results.

Leave SECTION 5 switched on. Those are SELECT statements, they only read, and they are safe to run as often as you like. Keep the commented out sections in the file. They are how you rebuild from nothing next time you sit at a wiped PC: select them, press **Ctrl and forward slash** again, and they come back to life.

Part 4. Chapter 5, sections 5.1 to 5.6

Six short steps, one for each section of Chapter 5. Add each one to the bottom of **students_db_backup.sql**, under the heading already waiting for it, and run it before you move on.

> **Write a comment above every statement. This is graded.**
>
> Above each statement you add, put a line comment in **your own words** saying what it does. Not a copy of the heading. Something like:
>
> ```
> -- Makes a copy of STUDENT with only four of the columns
> ```
>
> One point of this lab is for the file to still make sense to you in three weeks, when you have forgotten the lab but still have the file.

Step 11. Section 5.1, look at your schema

A schema is the container holding your tables. These two statements tell you which schema you are in and what is inside it.

```
SELECT DATABASE() AS my_schema;

SHOW FULL TABLES;
```

The first should say **students_db**. The second is worth remembering. Plain **SHOW TABLES** lists your views mixed in with your tables and gives you no way to tell which is which. Adding **FULL** adds a Table_type column that says BASE TABLE or VIEW.

Step 12. Section 5.2, look at the data types

```
DESCRIBE STUDENT;
```

Compare what comes back against why each type was chosen:

Column	Type	Why that type
student_id	int	A whole number with no decimal part, so INT
first_name	varchar(30)	Text of varying length, up to 30 characters
last_name	varchar(30)	Same, and the one you will widen in step 14

Column	Type	Why that type
major	varchar(40)	Longer text, because some major names are long
gpa	decimal(3,2)	1 digit before the point and 2 after, which is exactly the shape of a grade average. DECIMAL holds it exa
enrolled_on	date	A real date, so the database can compare and sort it properly

The Null column tells you which columns may be left empty, and the Key column marks the primary key with PRI.

Step 13. Section 5.3, create a table by copying it

You can build a new table out of a SELECT statement instead of listing the columns yourself. The SELECT is the subquery, and the new table gets one column for each column the SELECT returns.

```
CREATE TABLE STUDENT_COPY AS
SELECT student_id, first_name, last_name, gpa
FROM STUDENT;

SELECT * FROM STUDENT_COPY;
```

Five rows and four columns, because the SELECT asked for four. The major and the enrollment date were left behind. Now look at what you actually got:

```
SHOW CREATE TABLE STUDENT_COPY;
```

> **The copy did not bring the primary key with it**
>
> Read the output carefully. There is no PRIMARY KEY line. Copying a table this way brings the columns and the rows, and leaves the constraints behind.
>
> So STUDENT_COPY will happily accept two students with the same student_id, which the real STUDENT table would refuse. A table that looks right and has no constraints is worse than no table, because it quietly collects bad data. Step 14 puts the key back.

Step 14. Section 5.4, change the structure of the copy

ALTER TABLE changes a table that already has rows in it. Three things here: add a column, widen a column, and add the missing constraint.

```
-- add a new column on the end
ALTER TABLE STUDENT_COPY ADD COLUMN honors CHAR(1);

-- make an existing column wider
ALTER TABLE STUDENT_COPY MODIFY COLUMN last_name VARCHAR(50) NOT NULL;

-- put back the primary key the copy did not inherit
ALTER TABLE STUDENT_COPY ADD PRIMARY KEY (student_id);

DESCRIBE STUDENT_COPY;
```

Now prove the constraint took effect. This should fail:

```
INSERT INTO STUDENT_COPY VALUES (1,'Duplicate','Row',0.00,NULL);
```

You get **error 1062, duplicate entry 1 for key PRIMARY**, the same error from step 9. That is the proof. Before step 14 that row would have been accepted.

> **Watch out when you MODIFY a column**
>
> MODIFY COLUMN replaces the whole definition rather than adjusting part of it. If you leave out **NOT NULL** when you only meant to change the width, you have quietly removed the NOT NULL rule and the column will start accepting empty values.
>
> That is why the statement above repeats NOT NULL. Always write out the full definition you want, not just the part you are changing.

Last, remove the column you added:

```
ALTER TABLE STUDENT_COPY DROP COLUMN honors;
```

DROP COLUMN takes the column and everything in it, with no confirmation and no undo. Here it is a column you added a minute ago with nothing in it, so there is nothing to lose, which is the only sort of situation to practise it in.

Step 15. Section 5.5, insert, update and delete a row

Three statements that change data rather than structure. Run them one at a time and look at the row in between, so you can see each one land.

```
-- add a student
INSERT INTO STUDENT_COPY VALUES (6,'Omar','Faruqi',3.10);
SELECT * FROM STUDENT_COPY WHERE student_id = 6;

-- change that student's grade average
UPDATE STUDENT_COPY SET gpa = 3.35 WHERE student_id = 6;
SELECT * FROM STUDENT_COPY WHERE student_id = 6;

-- remove that student
DELETE FROM STUDENT_COPY WHERE student_id = 6;
SELECT COUNT(*) AS rows_left FROM STUDENT_COPY;
```

> **The WHERE clause is the whole safety net**
>
> Both UPDATE and DELETE above have a WHERE clause. Leave it off an UPDATE and you change **every row in the table**, so every student in the file ends up with the same grade average. Leave it off a DELETE and you empty the table.
>
> There is no confirmation prompt and no undo. The habit that saves you: run your WHERE clause as a SELECT first and look at what comes back. If the SELECT returns the rows you meant, the UPDATE or DELETE will hit those same rows.

Step 16. Section 5.6, create a view and update through it

The starter file already made you a view and you read from it in step 8. This time you write through one.

```
CREATE VIEW HIGH_GPA AS
SELECT student_id, first_name, last_name, gpa
FROM STUDENT_COPY
```

```
WHERE gpa >= 3.50;

SELECT * FROM HIGH_GPA ORDER BY gpa DESC;
```

Two students come back, the two at 3.50 or above. Now check the real table, change a value **through the view**, and check the real table again:

```
-- before
SELECT student_id, gpa FROM STUDENT_COPY WHERE student_id = 1;

-- this UPDATE names the VIEW, not the table
UPDATE HIGH_GPA SET gpa = 3.95 WHERE student_id = 1;

-- after
SELECT student_id, gpa FROM STUDENT_COPY WHERE student_id = 1;
```

Ana's grade average goes from 3.75 to 3.95 in **STUDENT_COPY**, even though your UPDATE never mentioned that table. That is an updatable view: the view holds no data of its own, so a change made through it goes straight to the table underneath.

Not every view works that way. Try this one:

```
CREATE VIEW GPA_SUMMARY AS
SELECT COUNT(*) AS n, ROUND(AVG(gpa), 2) AS avg_gpa
FROM STUDENT_COPY;

UPDATE GPA_SUMMARY SET avg_gpa = 4.00;
```

That one is refused with **error 1288, the target table GPA_SUMMARY of the UPDATE is not updatable**. The view's single row was worked out from five rows, so there is no one row underneath for the database to change. A view you can write through has to point at particular rows, which rules out anything built with an aggregate like COUNT or AVG.

Part 5. Put your file on GitHub

GitHub gives your file a web address. That address is what you hand in, and it is the only thing you hand in, so take your time over these six steps.

Save your file first. Press **Ctrl and S** in VS Code and check that the OneDrive icon shows a tick.

Step 17. Set up VS Code to work with GitHub

Even though you are going to use the website in a moment, set this up now. It is how you will work on your own machine, and it takes two minutes.

First, check that Git is installed. At the command prompt:

```
git --version
```

If it prints a version number, Git is there. If it says **'git' is not recognized**, Git is not installed on this PC and nothing in VS Code can push to GitHub. That is not a problem for handing this lab in, because the website route in step 19 does not need Git at all. Carry on reading so you know how it works, then use the website.

Second, install the extension.

1. Click the Extensions icon in the left bar, or press **Ctrl and Shift and X**.
2. Search for **GitHub Pull Requests**. The one you want is published by **GitHub**.
3. Click **Install**.

Third, sign in to your GitHub account from inside VS Code.

1. Look at the very bottom of the left bar for the **Accounts** icon, the little person shape just above the gear.
2. Click it and choose the option to sign in to GitHub. Depending on your version the wording is **Sign in to GitHub** or **Sign in with GitHub**.
3. Your browser opens a GitHub page asking you to authorize Visual Studio Code. Sign in to GitHub if it asks, then click the green **Continue** or **Authorize** button.
4. The browser then asks to open Visual Studio Code again. Allow it. The browser tab can be closed.
5. Back in VS Code, click the Accounts icon again. Your GitHub username should now be listed there. That is how you know it worked.

> **This sign in does not survive the night**
>
> Authorizing VS Code stores a credential on this PC. The PC is wiped tonight, so you will sign in again next week. That is one more reason the lab hands in a web address rather than anything stored locally.
>
> On your own laptop you do this once and it stays.

Step 18. The two ways to get a repository, and which to use today

A repository, usually shortened to repo, is one project folder on GitHub. There are two ways to make one and they are genuinely different, not two buttons for the same thing.

	On github.com in a browser	In VS Code with the extension
Who makes the repo	You, by filling in a form	VS Code, with one command called Publish to GitHub
Needs Git installed	No	Yes
Needs a sign in stored on the PC	No	Yes, and it is wiped tonight
How the file gets up there	You choose the file and upload it	VS Code pushes the whole folder at once
Afterwards	You edit and upload through the browser	You commit, push, pull and review pull requests without leaving the e
Best for	A lab PC that gets wiped nightly	Your own laptop

How the VS Code way works, so you recognise it later. With a folder open and Git installed, you click the **Source Control** icon in the left bar, the one that looks like a branching line. It offers **Publish to GitHub**. You pick a name and whether the repo is public or private, and VS Code creates it on GitHub and pushes every file in the folder in one action. There is no browser form to fill in and nothing to upload by hand.

What the Pull Requests part is actually for. Creating repos is the small part of what that extension does. Its real job is the **Pull Requests and Issues** view it adds to the sidebar, which lets you open someone else's proposed change, read it beside your own code, leave comments on particular lines, and approve it, all inside the editor. That is how changes get reviewed on a team. You will not need it for this lab, but it is why the extension exists and it is worth having seen.

Use the website for this lab

Everything about the VS Code route depends on things stored on this machine: Git, and the credential from step 17. Both are gone tomorrow.

The website route stores nothing locally, needs no install, and produces exactly the same repository and exactly the same web address. So that is what steps 19 and 20 do. Use the VS Code route at home, where it saves you real time.

One thing to check before you upload anything

Your folder contains a hidden folder called **.vscode** holding **settings.json**, which is where SQLTools saved your database connection. If you chose to save the password rather than be asked for it each time, **your password is sitting in that file in plain text**.

Open it and look. If the password is in there, do not put that file on GitHub. A public repo is readable by anyone.

This is why step 20 has you upload only your .sql file and nothing else. It is also the most common way people accidentally publish a password, and it is worth forming the habit now on a password that does not matter.

Step 19. Create the repository on github.com

Open a browser and go to **github.com**. Sign in, or click **Sign up** and make a free account. If you sign up, GitHub emails you a code you have to type back in, so use an address you can read right now. Write your username down.

Then:

1. Click the **+** at the very top right of the page, next to your profile picture, then choose **New repository**.
2. In **Repository name**, type **cis4720-lab7b**.
3. Leave the description empty.
4. Choose **Public**. This matters more than anything else on the page, see the box below.
5. Tick **Add a README file**.
6. Click the green **Create repository** button at the bottom.

Public, not Private

A private repo cannot be opened by anyone but you. If you choose Private, the address you hand in shows your instructor a page saying the repository does not exist, and the lab reads as not submitted.

If you have already made it private, you can change it: **Settings** on the repo, scroll to the bottom, **Change repository visibility**.

There is nothing private in the file you are uploading. It is five made up students.

Step 20. Upload your file and commit it

You are now looking at your empty repo with just a README in it. Committing is how GitHub saves a change, along with a note about what the change was.

1. Click **Add file**, near the green Code button, then choose **Upload files**.
2. Click **choose your files**, then find **CIS4720_Lab7b** inside your OneDrive and pick **students_db_backup.sql**. You can also drag the file onto the page from File Explorer. Upload only that one file.
3. Wait for the file name to appear on the page. If it is not listed, it did not upload.

4. Scroll down to the **Commit changes** box and replace the default text with something that says what you did, such as **Add lab7b SQL file with Chapter 5 statements**.
5. Click the green **Commit changes** button.

The page goes back to your repo and **students_db_backup.sql** is now listed. Click it. You should see your statements and your comments. If the comments are missing, you uploaded an older copy, so save in VS Code and upload again.

Step 21. Copy the address, then check it the hard way

The address of your repo is in the browser bar and looks like this, with your own username in place of the example:

```
https://github.com/yourusername/cis4720-lab7b
```

Copy it. Then check it properly, because this is the single most common way to lose the marks for this lab:

1. Open a **private** or **incognito** browser window. In most browsers that is **Ctrl, Shift and N**.
2. Paste the address and press Enter.
3. If you can see your repo and your file while signed out, your instructor can too. If you get a **404** page, the repo is still private, so go back and fix it as described in step 19.

Step 22. Hand in the address

Paste the address into the Canvas submission box for this lab and submit. That address is the whole submission. There is no file to upload to Canvas.

How this is graded

Steps	What it shows	Points
1 to 3	Signed in to the OneDrive app, folder made on the PC, file in it	1.0
4 to 7	Database created at the command prompt, connected from VS Code, and you can comment a line and a block	1.0
8 to 10	The file ran, and CREATE and INSERT are commented out, not deleted	2.0
11 to 16	All six Chapter 5 statements present and working	3.0
all	A comment in your own words above every statement you added	1.0
17 to 18	VS Code signed in to GitHub, and you can say how the two routes differ	1.0
19 to 22	A public GitHub repo holding your file, and the address handed in	1.0
	Total	10.0

Your repository is the evidence for all of it, which is why the address is the only thing you hand in. A statement that is not in the uploaded file did not happen as far as grading goes, so upload the file after you finish the SQL and not before. The due date is listed on Canvas LMS.

Previous labs: in_class_lab6_instructions.pdf, in_class_lab_midterm.pdf and in_class_lab7_instructions.pdf.

Reference: Coronel, C. and Morris, S. (2027). Database Systems: Design, Implementation, and Management (14th ed.). Cengage Learning. ISBN 979-8214410241. Chapter 5, sections 5.1 to 5.6.